

Optimizarea interogărilor SQL folosind indecsi

Seminar 6





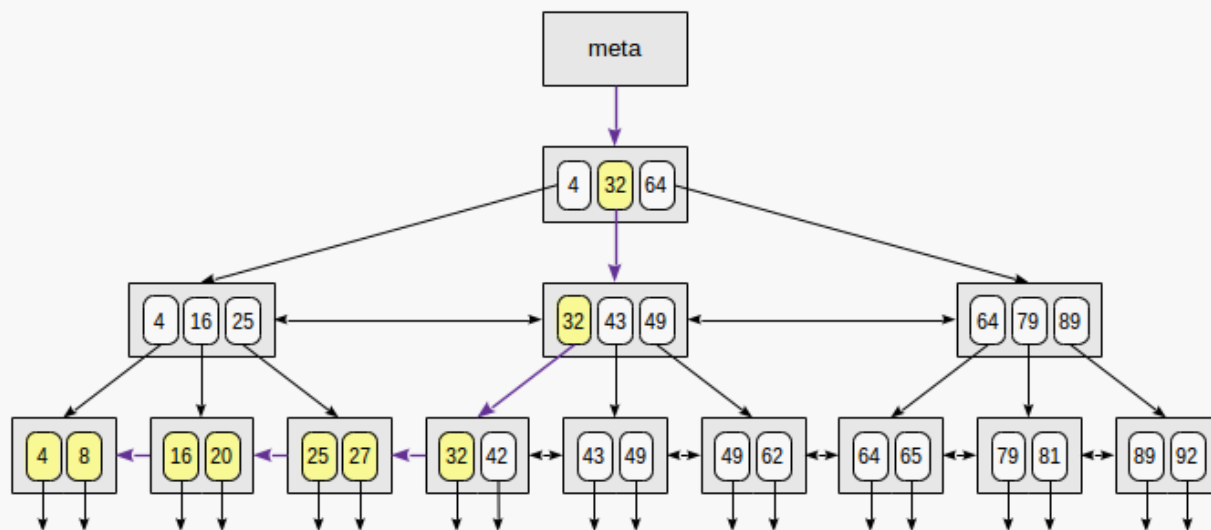
Indecși - quick recap

- Structură de date separată
- Accelerează accesul la operațiile de citire, dar încetinește scrierile
- INSERT/UPDATE/DELETE - trebuie să actualizeze toți indecșii

Tipuri de indecși suportati

Index Type	PostgreSQL	MySQL/InnoDB	SQL Server	Oracle	DB2
B-Tree	✓	✓	✓	✓	✓
Hash	✓	✓ (in-memory only)	✗	✗	✗
GiST	✓	✗	✗	✗	✗
GIN	✓	✗	✗	✗	✗
BRIN	✓	✗	✗	✗	✗
SP-GiST	✓	✗	✗	✗	✗
Bitmap	✓ (runtime only)	✗	✗	✓ (stored)	✓ (stored)
Full-text	✓ (GIN-based)	✓ (FULLTEXT)	✓	✓	✓
Spatial / R-Tree	✓ (PostGIS/GiST)	✓ (SPATIAL)	✓ (spatial)	✓ (SDO)	✓
Clustered	✗ (use CLUSTER)	✓ (always, InnoDB)	✓	✓ (IOT)	✓
Reverse key	✗	✗	✗	✓	✓
Function-based	✓ (expression index)	✓ (8.0+)	✓ (computed col)	✓	✓
Partial / Filtered	✓	✗	✓ (filtered)	✗	✗
Invisible / Unusable	✗	✓	✗	✓	✓
Descending	✓	✓ (8.0+)	✓	✓	✓
Composite	✓	✓	✓	✓	✓
Covering (INCLUDE)	✓ (v11+)	✓ (8.0+) ↓	✓	✗	✗

Indecșii B-Tree





Indecșii B-Tree

- Range Scan (Traversal + Horizontal Scan)
- Index compus (a, b, c) - este folosit doar pentru interogări care conțin a, a+b, a+b+c, dar nu pentru interogări care conțin doar b, c sau b+c
- Coloana cea mai selectivă la început

Exerciții - schema de lucru

```
CREATE TABLE Orders (  
  id INT PRIMARY KEY,  
  customer_id INT,  
  status VARCHAR(20), -- 5 valori distincte  
  created_at TIMESTAMP,  
  total DECIMAL(15,2),  
  email VARCHAR(50)  
);
```

Exercițiu 1

-- Query A:

```
SELECT * FROM Orders WHERE customer_id = 42 AND status = 'pending';
```

-- Query B:

```
SELECT * FROM Orders WHERE status = 'pending' ORDER BY created_at;
```

-- Query C:

```
SELECT customer_id, SUM(total)
FROM Orders
WHERE created_at BETWEEN '2025-01-01' AND '2025-12-31'
GROUP BY customer_id;
```

- Adăugați 1 sau 2 indecși pentru fiecare interogare
- Justificați ordinea coloanelor
- Este suficient un singur index compus pentru toate interogările ? Justificați

Exercițiu 2 – Index Killers

-- Tabela are un index pe coloana **name**. De ce nu este utilizat ?

```
SELECT * FROM Orders WHERE EXTRACT (YEAR FROM created_at) = 2026 ;
```

-- Tabela are un index pe coloana **name**. De ce nu este utilizat ?

```
SELECT * FROM Customers WHERE LOWER(name)='popescu';
```

-- Tabela are un index pe coloana **total**. De ce nu este utilizat ?

```
SELECT * FROM Orders WHERE total > 500 OR status='pending';
```

-- Similar, avem index pe coloana **email**, dar nu este folosit.

```
SELECT * FROM Orders WHERE email LIKE '%yahoo.com';
```

- Rescrieți interogările sau indecșii astfel încât indecșii să fie utilizați

Exercițiu 3

-- Interogarea urmatoare este executată de milioane de ori pe zi
`SELECT status, total FROM Orders WHERE customer_id = 42;`

- Index actual: `CREATE INDEX idx_cust ON Orders (customer_id);`
- `EXPLAIN` arată Index Scan dar tot încarcă pagini din heap
- Cum putem elimina citirile din paginile heap ?
- Care sunt consecințele soluției propuse ?

Exercițiu 4

- Tabela are 50 de milioane de înregistrări
 - 99% din comenzi au **status = 'completed'**.
 - 1% au **status = 'pending'**.
 - Aplicația citește comenzile **'pending'** de sute de ori pe secundă
 - Un index normal pe coloana status poate fi ignorat de către optimizator. De ce? Dar dacă căutăm comenzile **'cancelled'**.
 - Propuneți un index mai bun. Verificați cu EXPLAIN ANALYZE.
-

Exercițiu 5

Avem următoarele tabele:

- Orders (Id, Customer_Id, Status, Created_at, Total, Email)
- Customers (Id, Name)
- Products (Id, Category_Id, Name, Price)
- OrderItems (Id, Order_Id, Product_Id, Quantity, Price, Value)

Și următoarea distribuție de operațiuni:

- 80% citiri: istoric comenzi per client, detalii comenzi istorice, căutare produse după categorie
- 20% scrieri: comenzi noi inserate rapid, actualizări de status comenzi

Propuneți o schemă (set de indecși) și specificați care indecși impactează performanța la scriere.

Exercițiu 5 - soluție

1. `CREATE INDEX idx_orders_customer_id_created_total ON Orders (customer_id, created_at DESC, total);`
2. `CREATE INDEX idx_order_items_order_id ON OrderItems (order_id);`
3. `CREATE INDEX idx_order_items_product_id ON OrderItems (product_id);`
4. `CREATE INDEX idx_products_cat_status_covering ON Products (category_id) INCLUDE (name, price);`